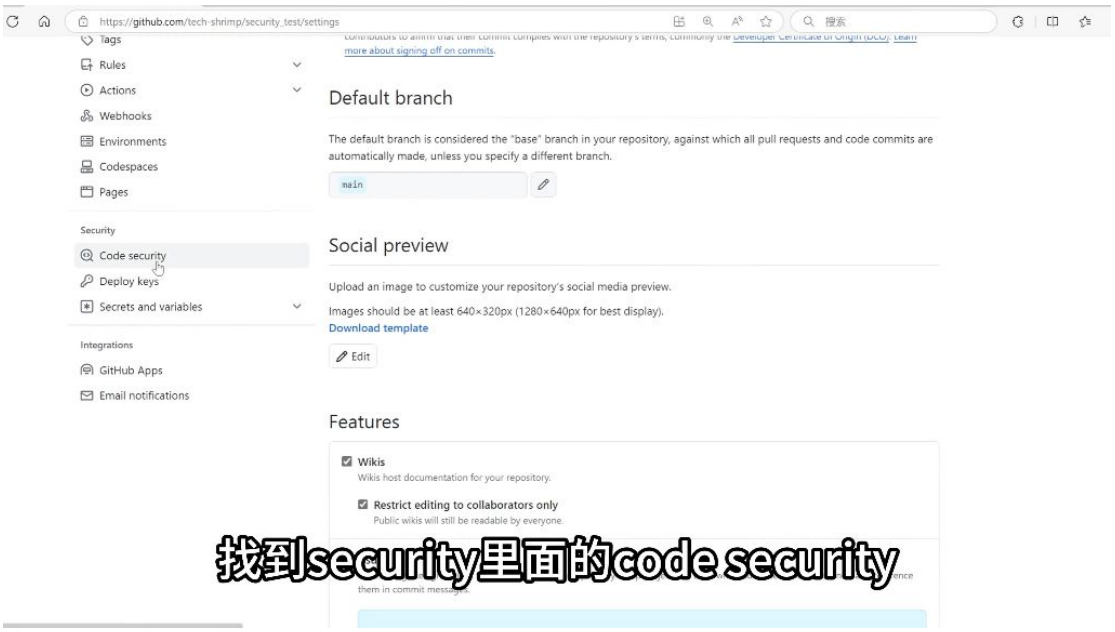
-- 以下为AI生成的图文笔记内容 --

一、GitHub的高级安全功能00:00

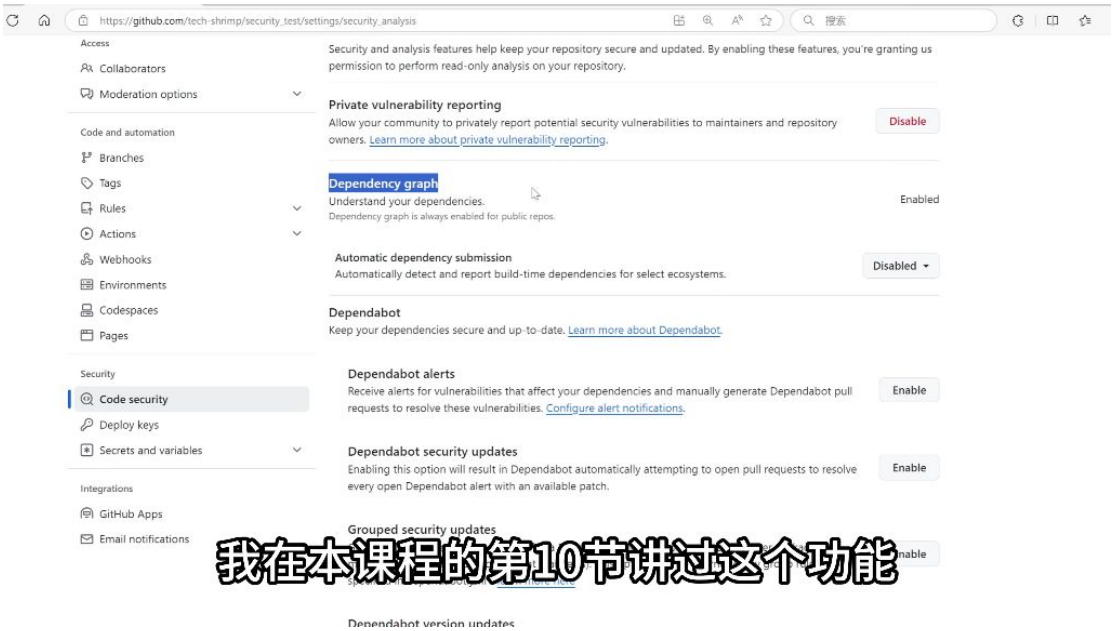
1. 安全 00:16

1）安全漏洞的隐私报告 00:22



-
- **功能入口**: 在仓库Settings → Security → Code security中开启Private vulnerability reporting功能
- **作用机制**:
  - 允许其他用户秘密报告仓库可能存在的安全风险
  - 避免在公开的Issues界面暴露安全问题而被恶意利用
  - 报告内容仅管理员和报告者可见，不会公开在网上
- **报告流程**:
  - 用户点击Security → Report a vulnerability
  - 填写漏洞详情（如问题代码行号、描述等）
  - 管理员在Security → Advisories查看报告

2）依赖图 01:24

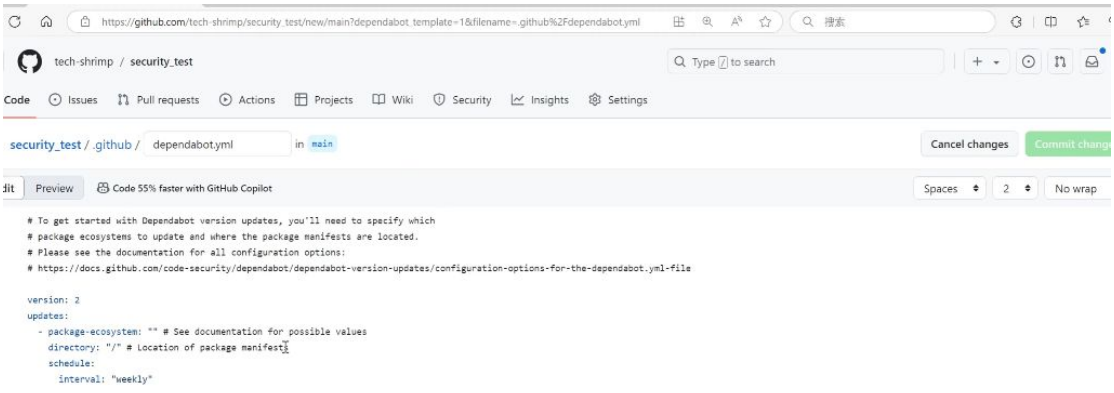


我在本课程的第10节讲过这个功能

-
- **核心功能**: 可视化展示项目依赖的第三方库
- **自动依赖提交**:
  - 通过GitHub Actions增强依赖扫描能力
  - 目前仅支持Maven项目
  - 对于Maven项目建议开启以获取完整依赖分析

3）Dependabot 02:02

- 操作演示 02:16



这里自动生成了一个配置文件

-
  - **启用步骤**:

- 进入Insights → Dependency Graph
- 点击Enable Dependabot → Create config
- 修改package-ecosystem为项目使用的包管理器（如pip）
- 设置扫描频率（默认weekly可改为daily）

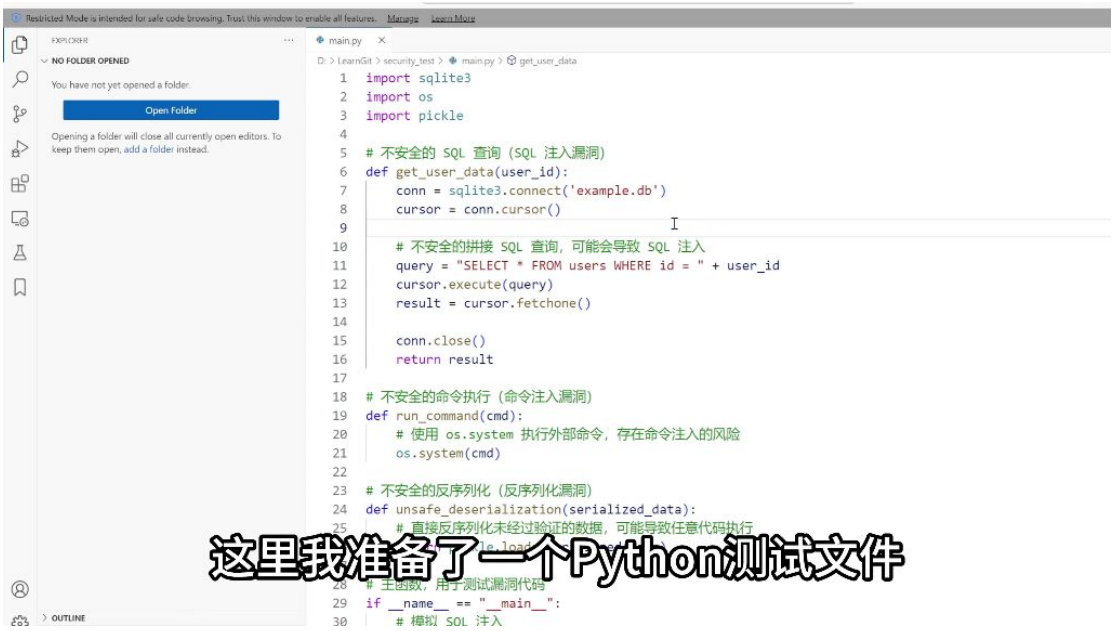
○ **工作流程**:

- 自动扫描项目依赖的安全问题
- 发现漏洞后创建PR自动升级版本
- 示例：将fastapi从0.100.0升级到0.115.0修复漏洞

○ **进阶配置**:

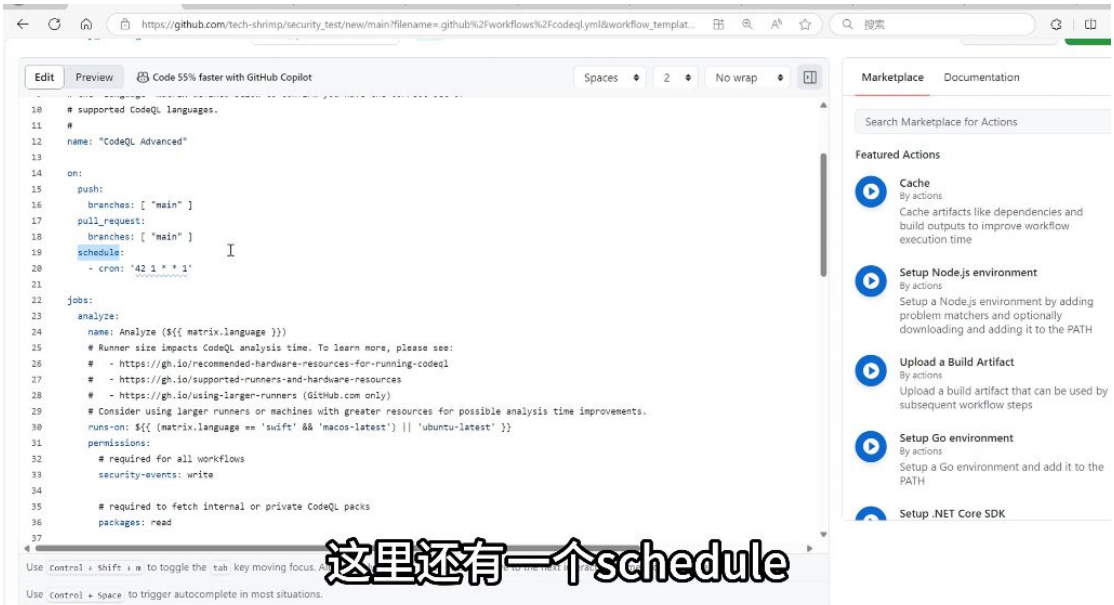
- **安全警报**: 开启后每日扫描发现问题会发送通知
- **自动PR**: 发现安全问题自动创建修复PR
- **分组更新**: 将多个依赖更新合并到一个PR
- **版本更新**: 可选是否包含非安全性的版本更新

4）代码扫描 05:29

- 操作演示 05:47
- 

- **测试用例**:
  - 不安全的SQL查询（SQL注入漏洞）
  - 不安全的命令执行（命令注入漏洞）
  - 不安全的反序列化（反序列化漏洞）
  - 使用MD5哈希密码（弱加密算法）
- **扫描结果**:
  - 在Security → Code scanning查看扫描出的漏洞
  - 点击具体漏洞查看详情和修复建议
  - 示例：检测到MD5哈希建议改用Argon2

5）CodeQL 07:02

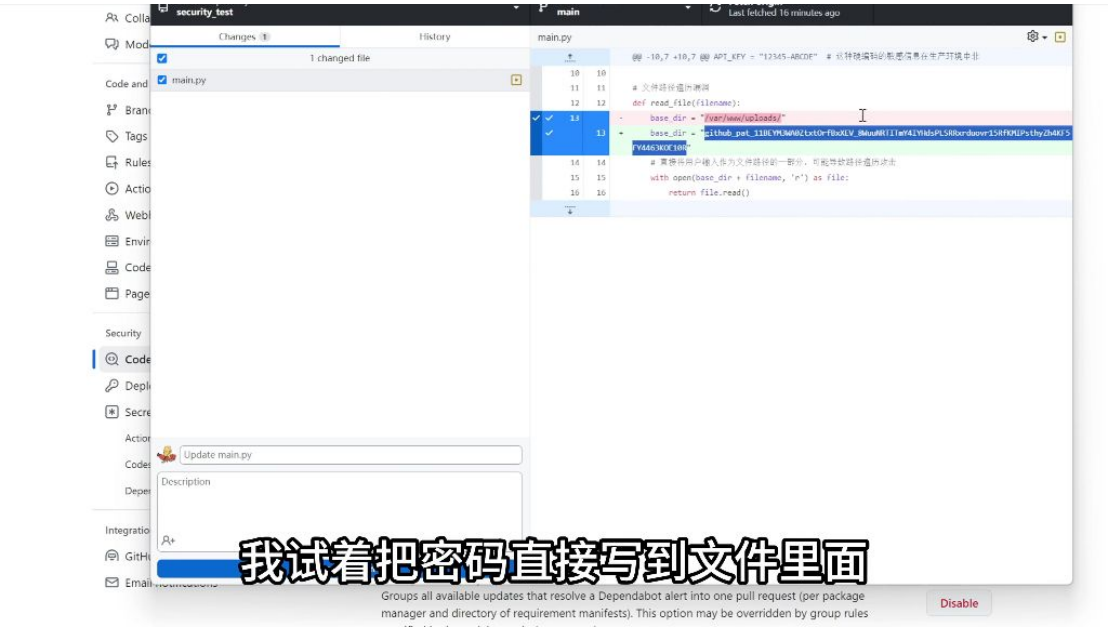
- 操作演示 07:38
- 

  - **高级配置**:
    - 自动生成GitHub Actions工作流文件
    - 可自定义扫描分支和定时任务（如每周一1:42执行）
  - **保护规则**:

- 设置合并代码时的安全检查阈值
- 可选择阻止合并的漏洞严重级别（Critical/High/Medium）
- 与分支规则集配合使用，确保代码质量

6）Secrets 08:34

- 操作演示 08:49



-
  - **保护机制**:
    - 自动检测代码中的密钥、密码等敏感信息
    - 发现密钥后会阻止push操作
    - 必须从代码中移除密钥才能继续提交
  - **正确做法**:
    - 将密钥保存在Settings → Secrets and variables中
    - 绝对不要将密钥硬编码在代码或配置文件中

2. 部署密钥 09:54

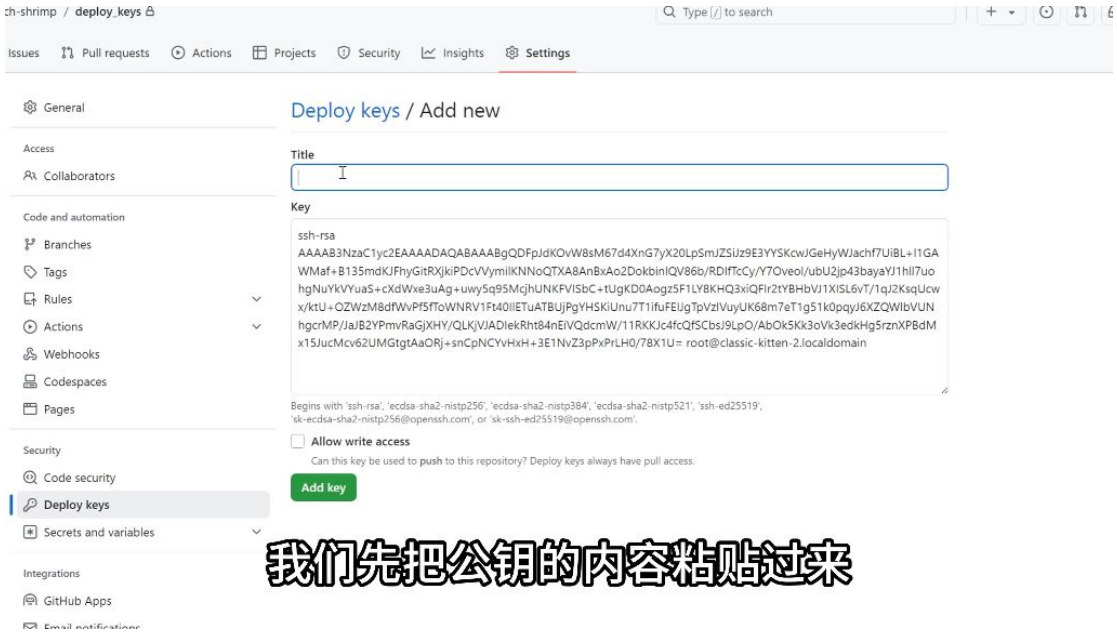
1）服务器配置公/私钥 10:25

- **生成密钥对**:

ssh-keygen -t rsa

- 密钥默认保存在/root/.ssh/目录
- 包含私钥id_rsa和公钥id_rsa.pub

2）GitHub配置公钥 10:58



- **配置步骤**:
  - 复制公钥内容（cat id_rsa.pub）
  - 在GitHub仓库Settings → Deploy keys
  - 点击Add deploy key粘贴公钥
  - 设置密钥名称和访问权限（读写/只读）
- **多服务器配置**:
  - 每台服务器需要单独生成密钥对
  - 在GitHub中添加对应的公钥
- **克隆私有仓库**:

git clone git@github.com:user/repo.git

  - 首次连接需确认主机指纹（输入yes）

二、知识小结

知识点	核心内容	关键操作/配置	安全等级
Private Vulnerability Reporting	允许用户秘密报告仓库安全风险	在Security > Code Security开启开关	高
Dependency Graph	显示项目第三方依赖关系	支持Maven项目的Automate Dependency Submission	中
Dependabot	自动化依赖项安全扫描与修复	配置package ecosystem(如pip/npm)和扫描频率	**高**
Dependabot进阶配置	1. 安全通知 2. 自动创建PR 3. 合并多个PR 4. 非安全更新	在Code Security调整: - Alert notifications - Auto-PR creation - Group updates	可定制
CodeQL分析	扫描代码本身的安全漏洞	1. 启用Code Scanning 2. 查看Security > Code Scanning结果 3. 通过AI生成修复建议	**最高**
分支保护规则	合并代码时的安全检查阈值	设置critical/media级别漏洞拦截规则	高
Secret Scanning	检测代码中的密钥泄露	自动拦截含密钥的push操作	**最高**
Deploy Key配置	服务器访问私有仓库的SSH认证	1. 生成SSH密钥对 2. 在仓库/账户设置添加公钥	中
FastAPI漏洞示例	旧版本(1.0.0)存在安全风险	Dependabot自动升级到1.1.5的PR演示	高
MD5哈希漏洞	使用不安全的哈希算法	CodeQL检测后通过AI建议替换为SHA-256	**最高**